

Customizing AI Workflows Cheat Sheet

Irurueta Docs › Guides & References › AI › Customizing AI Workflows

Baseline: VS Code 1.139 with the GitHub Copilot Chat extension (the single extension that now ships every Copilot feature), GitHub Copilot CLI 1.0.88 (@github/copilot) and Claude Code 2.1.283. Re-verified as current on 2026-09-28. Worked example: ai-catalog (102 iru-* skills, 3 agents) at commit 58fb709. Quick reference only — verify against the official docs before production use.

WHAT TO USE

If you need...	Use
always-on rules	instructions
a reusable procedure	skill or prompt file
an isolated worker	subagent
deterministic enforcement	hook
distribution	plugin
external capability	MCP

SKILL-TESTING CHECKLIST

- Trigger-accuracy tests (should/shouldn't fire)
- Dry runs in a temp checkout
- Golden-output checks
- Regression suites when editing skills
- Versioning and a changelog for skills
- At least three evaluations created
- Tested with Haiku, Sonnet, and Opus
- Tested with real usage scenarios

FILE LOCATIONS

In Claude Code

- ./CLAUDE.md / ~/.claude/CLAUDE.md — project instructions
- ~/.claude/CLAUDE.md — user; ./CLAUDE.local.md — personal (gitignore it)
- .claude/skills/<name>/SKILL.md — project; ~/.claude/skills/ — personal
- .claude/agents/<name>.md — project agents; ~/.claude/agents/ — user
- .claude/commands/<name>.md — commands (older format); \$ARGUMENTS, 0-based \$0 / \$1
- .claude/settings.json / .claude/settings.local.json — settings
- .mcp.json — project MCP servers
- .claude-plugin/plugin.json — plugin manifest

In GitHub Copilot (VS Code)

- .github/copilot-instructions.md — repo instructions
- .github/instructions/<name>.instructions.md (applyTo: glob) — path-scoped
- AGENTS.md — open format; Claude Code reads it if no CLAUDE.md
- .github/skills/ or .agents/skills/ — project skills (also reads .claude/skills/); ~/.copilot/skills/ / ~/.agents/skills/ — personal (VS Code also ~/.claude/skills/)
- .github/prompts/*.prompt.md — prompt files
- .github/agents/*.agent.md — custom agents
- .vscode/mcp.json — MCP servers

PLUGIN LAYOUT

```
my-plugin/
├── .claude-plugin/
│   └── plugin.json
├── skills/<n>/SKILL.md
├── agents/<n>.md
├── hooks/hooks.json
└── .mcp.json
```

SECURITY CHECKLIST

- Trusting third-party skills, plugins and MCP servers (blast radius differs)
- Prompt injection via tool output and files — treat as data, never instructions
- No hardcoded secrets in hooks and settings
- Least-privilege allowed-tools
- Review plugin contents before install — hooks/MCP run outside the sandbox
- Managed settings for organisations

SKILL.MD & AGENT FRONTMATTER

SKILL.md keys

- name, description, when_to_use
- argument-hint, arguments
- disable-model-invocation, user-invocable
- allowed-tools, disallowed-tools
- model, effort
- context: fork + agent, background
- hooks, paths

- shell, metadata, license, compatibility

Agent .claude/agents/*.md keys

- name, description
- tools, disallowedTools
- model, color, effort
- permissionMode, isolation
- skills, mcpServers, hooks
- maxTurns, memory, background
- omitClaudeMd, initialPrompt, experimental

HOOK EVENTS & EXIT CODES

Can block: PreToolUse, UserPromptSubmit, Stop, SubagentStop, PreCompact.

Observe only: PostToolUse (tool already ran), PostToolUseFailure, SessionStart, SessionEnd, Notification, PostCompact

Exit	Meaning
0	No objection (normal permission flow still applies); JSON on stdout for structured control
2	Blocks on events that can block; stderr becomes the reason fed back
other	Non-blocking error; action proceeds; valid JSON on stdout is still read
JSON	PreToolUse: hookSpecificOutput.permissionDecision allow deny ask; Stop: "decision": "block" + reason

PERMISSION RULE SYNTAX

- Bash (mvn *) — any mvn command, including bare mvn
- Read (./secrets/**) — as deny/ask: a secrets dir at any depth; as allow: only <cwd>/secrets
- mcp__<server>__<tool> — one tool of one configured MCP server
- WebFetch (domain:example.com) — exact hostname; domain:*.example.com — subdomains
- Evaluation order: **deny, then ask, then allow** — first match wins
- Modes: default, acceptEdits, plan, auto, dontAsk, bypassPermissions
- VS Code picker: Manual (default), Assisted, Allow all; Autopilot is an agent mode

12 OPTIMISATION TECHNIQUES

- Context budget awareness
- Progressive disclosure via reference files and routing tables
- Context isolation with compact structured summaries
- Baselines, reporting only regressions
- Validating once per group
- Parallel fan-out
- Model tiering and effort levels
- Scoped allowed-tools
- Deterministic work in scripts or hooks rather than prose
- Idempotency, checkpointing and resumability
- Avoiding re-prompting (pass-through args)
- Prompt-cache-friendly structure (stable prefixes)

OUTPUT STYLE, STATUS LINE, KEYS, ENV

- Output style** (in the system prompt): /config › Output style, or "outputStyle": "Explanatory"; built-ins Default, Proactive, Concise, Explanatory, Learning; custom: .claude/output-styles/*.md (keep-coding-instructions: true)
- Status line** (never enters context): "statusLine": {"type": "command", "command": "~/.claude/statusline.sh"}; gets session JSON on stdin; /statusline <what to show> generates one
- Keybindings:** /keybindings → ~/.claude/keybindings.json: {"context": "Chat", "bindings": {"ctrl+e": "chat:externalEditor"}}
- Env vars:** "env": {"BASH_DEFAULT_TIMEOUT_MS": "300000"} in settings

COPILOT HOOKS

- .github/hooks/*.json — repo hooks (the only ones a cloud-agent job has); ~/.copilot/hooks/*.json — personal, CLI
- Events:** sessionStart, sessionEnd, userPromptSubmitted, preToolUse, postToolUse, agentStop, subagentStop, errorOccurred
- {"version": 1, "hooks": {"preToolUse": [{"type": "command", "bash": "~/.check.sh"}]}; also powershell, cwd, env, timeoutSec (default 30)
- preToolUse **prints** {"permissionDecision": "deny", "permissionDecisionReason": "..."} (allow|deny|ask); fails closed on error, open on timeout

MCP SERVERS

- claude mcp add --transport http <name> <url> — remote; claude mcp add <name> -- <cmd> [args] — stdio
- scope local (default, ~/.claude.json, this project) | project (.mcp.json, shared) | user (~/.claude.json, all projects)
- claude mcp list / get <name> check status; /mcp signs in to OAuth servers
- Tool names: mcp__<server>__<tool>; plugin servers: mcp_plugin__<plugin>__<server>__<tool> — use in rules, tool lists, hook matchers
- Copilot: .vscode/mcp.json (servers) or portable .mcp.json (mcpServers) in VS Code; CLI /mcp add → ~/.copilot/mcp-config.json

ORCHESTRATION PATTERNS

- Skill → skill:** Skill({skill: "iru-plan", args: "<id>"}) — same context
- Skill → agent:** Agent({subagent_type: "iru-isolated-skill-executor"}) — fresh context; only a summary returns
- Dispatch by key:** glob installed *-code-one-task-group skills to pick the language skill
- Fan-out / fan-in:** a group's Agent() calls in one response; gate-runner validates once
- Durable state:** plan checkboxes, its Base branch: line, .archive/
- Live tasks:** TaskCreate / TaskUpdate; newer models opt in via env var

COPILOT .AGENT.MD & .PROMPT.MD KEYS

.github/agents/*.agent.md
description, name, argument-hint, tools, agents, model (name or ordered list), user-invocable, disable-model-invocation, target (vscode|github-copilot), mcp-servers, handoffs (label, agent, prompt, send, model), hooks (Preview)

.github/prompts/*.prompt.md
description, name, argument-hint, agent (ask|agent|plan|<custom>), model, tools (<server>/* = all of an MCP server's tools)